BRÚJULA ACADÉMICA

Plataforma web semántica de recomendaciones académicas para estudiantes

PROYECTO FINAL DE CURSO

Curso: Ingeniería Web | Periodo: 2620

Universidad Cooperativa de Colombia | Facultad de Ingeniería

Integrantes: [Completar nombres del equipo]

Docente: [Completar nombre del docente]

Fecha: Octubre de 2026

Estado del documento

Propuesta conceptual para revisión y aprobación del docente. El tema aún no se presenta como aprobado. Los registros, estudiantes, perfiles, puntajes y recursos de ejemplo son sintéticos. No se afirma haber ejecutado todavía el razonador de Protégé ni las consultas en una instancia real de Neo4j.

Ficha del proyecto

Elemento	Definición
Nombre	Brújula Académica
Dominio	Acompañamiento y orientación académica en educación superior
Problema que aborda	Dificultad para encontrar recursos de aprendizaje apropiados según intereses, competencias por fortalecer, nivel y tiempo disponible.
Solución propuesta	Plataforma web conceptual que construye recomendaciones explicables a partir de perfiles, un catálogo semántico y relaciones de conocimiento.
Usuarios	Estudiante, asesor académico y coordinador/curador de contenidos.
Enfoque principal	Ingeniería Web, modelado conceptual y navegacional, REST, ontología OWL y grafo Neo4j.
Alcance	Diseño y prototipo navegable; modelo semántico y consultas demostrables con datos sintéticos.
Fuera de alcance	Integración real con el sistema de información académica, decisiones sobre matrícula/notas, autenticación productiva y uso de datos personales reales.

Resumen ejecutivo

Brújula Académica propone una solución web para orientar a estudiantes hacia recursos, cursos cortos y actividades de práctica relacionados con sus intereses y con las competencias que desean fortalecer. La propuesta no se limita a crear un catálogo: representa explícitamente las relaciones entre estudiantes, perfiles, temas, competencias, recursos, cursos, recomendaciones y retroalimentación, de modo que el sistema pueda explicar el motivo de cada sugerencia y descubrir asociaciones que no estén guardadas como recomendación directa.

La solución combina un prototipo web responsive, una arquitectura por capas, servicios REST, una operación SOAP/WSDL de referencia, una ontología OWL abierta en Protégé y un grafo Neo4j consultable mediante Cypher. Se adopta UWE (UML-based Web Engineering) como marco de modelado, porque permite separar el modelo conceptual del dominio, la navegación y la presentación; la propuesta se valida mediante revisión técnica y evaluación heurística inicial, pendiente de pruebas con usuarios reales.

Idea central
La plataforma recomienda recursos de aprendizaje y explica el porqué. No califica, etiqueta ni toma decisiones académicas de alto impacto sobre el estudiante.

Contenido

- 1. Introducción y formulación del problema
- 2. Objetivos, alcance y usuarios
- 3. Requisitos y reglas del dominio
- 4. Fundamentación académica
- 5. Metodología de Ingeniería Web
- 6. Arquitectura web
- 7. Modelado conceptual, casos de uso y navegación
- 8. Prototipo y usabilidad
- 9. Diseño de servicios web REST y SOAP/WSDL
- 10. Ontología del dominio en OWL
- 11. Grafo de conocimiento en Neo4j
- 12. Integración, trazabilidad y evaluación
- 13. Seguridad, privacidad y uso responsable
- 14. Plan de trabajo y criterios de aceptación
- 15. Conclusiones y trabajo futuro
- Referencias
- Anexos: instrucciones para ejecutar las evidencias

1. Introducción y formulación del problema

1.1 Contexto

En un entorno universitario, los estudiantes encuentran tutoriales, cursos abiertos, guías, laboratorios y material complementario en diferentes sitios. La abundancia de recursos no garantiza que puedan identificar cuál se ajusta mejor a sus intereses, nivel de dominio, cursos actuales o tiempo disponible. Un listado por palabras clave suele mostrar resultados sin explicar su relación con una necesidad formativa concreta.

Los sistemas de recomendación educativos se han investigado como instrumentos para filtrar contenido y apoyar actividades de enseñanza y aprendizaje. Una revisión sistemática de la literatura identifica tendencias, límites y oportunidades de estos sistemas, lo que respalda diseñar recomendaciones útiles sin confundir una coincidencia computacional con una decisión pedagógica definitiva (Silva et al., 2023).

1.2 Descripción del problema

El problema consiste en la falta de una vista integrada y explicable que conecte los intereses declarados por un estudiante con recursos que fortalezcan competencias concretas y que además indique el nivel, duración, modalidad y razón de la sugerencia. Para los asesores, la ausencia de una representación común dificulta descubrir patrones agregados de temas que podrían atenderse con talleres o rutas de apoyo.

1.3 Pregunta orientadora

¿Cómo diseñar una solución web semántica que relacione perfiles, temas, competencias y recursos de aprendizaje para ofrecer recomendaciones explicables, consultables e integradas con un modelo de conocimiento?

1.4 Justificación de una solución web

- El acceso mediante navegador permite utilizar la propuesta desde distintos dispositivos sin instalar un programa de escritorio.
- Una API separa la interfaz de los servicios y permite que un prototipo futuro pueda conectarse a otras interfaces o catálogos.
- El modelo semántico y el grafo representan relaciones entre conceptos, no solo operaciones CRUD.
- La navegación puede organizarse por tareas del usuario: perfilar intereses, explorar sugerencias, revisar explicaciones y aportar retroalimentación.
- El diseño conceptual permite discutir usabilidad, privacidad y consistencia antes de financiar o desplegar un sistema productivo.

2. Objetivos, alcance y usuarios

2.1 Objetivo general

Diseñar una plataforma web semántica de recomendaciones académicas que relacione perfiles de estudiantes, competencias, temas y recursos de aprendizaje para generar sugerencias explicables y apoyar la orientación académica mediante modelos, servicios y consultas de conocimiento.

2.2 Objetivos específicos

1. Identificar el problema, los actores, las funcionalidades y las reglas del dominio académico.

2. Definir una arquitectura por capas y un diseño REST que separe presentación, lógica de recomendación, semántica y persistencia.
3. Construir modelos conceptual y navegacional, junto con un prototipo de interfaces principales.
4. Formalizar una ontología OWL con clases, propiedades, dominios, rangos, restricciones e individuos sintéticos.
5. Representar relaciones de estudiantes, recursos y competencias en Neo4j y proponer consultas Cypher, incluida una recomendación derivada.
6. Evaluar el diseño mediante heurísticas de usabilidad y definir criterios verificables para una implementación futura.

2.3 Alcance

Incluye	No incluye
Propuesta conceptual y técnica centrada en orientación académica.	Prometer mejoras reales de rendimiento académico sin un estudio empírico.
Prototipo navegable en HTML/CSS/JavaScript con datos de ejemplo.	Inicio de sesión real, despliegue, integración SSO o conexión institucional.
Especificación REST/OpenAPI y ejemplo de operación SOAP/WSDL.	Servicios productivos, SLA, secretos, certificados o infraestructura real.
Archivo OWL, script Cypher, preguntas de competencia y pruebas sugeridas.	Afirmar que se ejecutaron herramientas externas sin capturas de evidencia.
Datos ficticios y reglas explicables basadas en intereses, temas y competencias.	Usar datos sensibles, inferir diagnósticos personales o automatizar sanciones/decisiones.

2.4 Perfiles de usuario

Actor	Necesidades	Permisos propuestos
Estudiante	Configurar intereses, indicar objetivos, explorar recursos, entender el motivo de las sugerencias y dar retroalimentación.	Consultar y editar su propio perfil; ver catálogo y recomendaciones; guardar/ocultar recursos propios.
Asesor académico	Orientar rutas de aprendizaje y observar necesidades agregadas de acompañamiento.	Consultar información agregada autorizada; sugerir recursos; no editar datos de perfil sin permiso.
Coordinador/curador de contenidos	Mantener catálogo, temas, niveles, duración y relaciones con competencias.	Crear/actualizar recursos; validar metadatos; revisar cobertura del catálogo.
Administrador técnico (apoyo)	Gestionar disponibilidad, roles y auditoría técnica si hubiera implementación.	Administración de acceso y configuración; no se propone como actor central de la demostración.

3. Requisitos y reglas del dominio

3.1 Funcionalidades principales

ID	Requisito	Descripción / criterio
RF-01	Gestionar perfil académico	El estudiante define objetivos, intereses y competencias que desea fortalecer.
RF-02	Explorar catálogo	Buscar y filtrar recursos por tema, dificultad, modalidad y duración estimada.
RF-03	Generar recomendaciones	Combinar coincidencia temática, competencia objetivo, nivel y carga estimada.
RF-04	Explicar cada recomendación	Mostrar las relaciones que justifican la sugerencia en lenguaje comprensible.
RF-05	Gestionar retroalimentación	Permitir guardar, completar, ocultar o valorar un recurso.
RF-06	Consultar el dominio semántico	Relacionar cursos, competencias, temas y recursos en la ontología y el grafo.
RF-07	Ofrecer resumen de asesoría	Mostrar patrones agregados para planificar talleres o recursos, sin ranking individual.
RF-08	Mantener metadatos del catálogo	El curador administra recursos y sus competencias/temas asociados.

3.2 Requisitos no funcionales

ID	Atributo	Criterio de diseño
RNF-01	Usabilidad	Etiquetas en lenguaje claro, navegación consistente, respuesta visible a cada acción, errores comprensibles y control de salida.
RNF-02	Accesibilidad	Contraste adecuado, control por teclado, foco visible, etiquetas accesibles y estructura responsive; verificar con herramientas y personas usuarias.

ID	Atributo	Criterio de diseño
RNF-03	Seguridad	HTTPS, control por rol y por recurso, validación de entradas, protección de credenciales y registro de acciones en una implementación real.
RNF-04	Privacidad	Minimización de datos, consentimiento informado, propósito declarado, retención limitada y acceso restringido.
RNF-05	Mantenibilidad	Separación entre interfaz, servicios, reglas de recomendación, ontología y almacenamiento.
RNF-06	Explicabilidad	Cada recomendación debe registrar los criterios que contribuyeron al resultado.
RNF-07	Rendimiento objetivo	Como objetivo de diseño, responder en menos de 2 s en el percentil 95 con carga académica pequeña; verificar con pruebas, no asumirlo como medido.
RNF-08	Interoperabilidad	Contratos JSON para REST; interfaz SOAP/WSDL de referencia para un sistema legado.

3.3 Reglas de negocio

- RB-01. Una recomendación debe estar asociada a un estudiante y a uno o más recursos de aprendizaje.
- RB-02. Un recurso debe describir como mínimo título, tema, dificultad, modalidad y duración estimada antes de entrar al catálogo recomendado.
- RB-03. Un candidato aumenta su afinidad si coincide con intereses declarados y aborda una competencia que el estudiante desea fortalecer.
- RB-04. Una coincidencia no equivale a garantía de aprendizaje ni a evaluación de capacidad; se presenta como apoyo opcional.
- RB-05. Los filtros de nivel, modalidad y tiempo deben poder limitar los resultados sin ocultar el criterio aplicado.
- RB-06. Los datos de asesoría se muestran de forma agregada; la implementación real debe evaluar umbrales mínimos para evitar reidentificación.
- RB-07. El estudiante puede ignorar, guardar, ocultar o valorar una recomendación, conservando el control sobre su ruta.
- RB-08. Los recursos de demostración y perfiles del proyecto son sintéticos y no representan registros institucionales reales.

4. Fundamentación académica

La propuesta integra fundamentos de Ingeniería Web, modelado dirigido por modelos, arquitectura de servicios, diseño centrado en el usuario, recomendación educativa y representación semántica. Las fuentes se utilizan para justificar decisiones concretas, no como resúmenes aislados.

4.1 Ingeniería Web y modelado

Murugesan et al. (2001) presentan la Ingeniería Web como un enfoque sistemático y disciplinado para desarrollar, operar y mantener sistemas basados en web. Por ello se propone definir requisitos, modelos, arquitectura, pruebas y criterios de aceptación antes de considerar una implementación productiva. Hincapié Londoño y Duitama (2012) revisan métodos de ingeniería web dirigidos por modelos y sus formas de representar una aplicación. Esta revisión respalda mantener separados el dominio, la navegación y la presentación para reducir ambigüedades entre lo que el sistema conoce y lo que el usuario recorre.

4.2 Arquitectura de servicios

Fielding (2000) formaliza REST como un estilo arquitectónico construido sobre restricciones que favorecen interfaces uniformes, despliegue independiente e intermediarios. En consecuencia, los recursos del diseño se identifican con URI y se exponen mediante métodos HTTP explícitos, respuestas JSON y errores normalizados, en lugar de acoplar la interfaz a la estructura de la base de datos.

4.3 Recomendaciones educativas

Silva et al. (2023) revisan sistemas de recomendación educativos y discuten tendencias, limitaciones y oportunidades. La decisión de este proyecto es construir un recomendador basado en reglas semánticas y transparentes, con posibilidad de evaluación futura, antes que presentar un algoritmo opaco como si fuera validado. Como señalan revisiones del campo, la evaluación del sistema debe contemplar relevancia educativa y no solamente clics o preferencias.

4.4 Usabilidad y diseño centrado en las personas

Las heurísticas de Nielsen ofrecen principios generales para evaluar la interacción, como visibilidad del estado, correspondencia con el mundo real, control del usuario, consistencia y prevención de errores (Nielsen, 1994/2024). La norma ISO 9241-210:2019 fundamenta la participación de usuarios y la consideración de necesidades humanas durante el ciclo de vida. Por ello la evaluación incluida en esta propuesta se declara heurística e inicial: no sustituye pruebas de usabilidad con estudiantes y asesores reales.

4.5 Ontologías y grafos

Noy y McGuinness (2001) describen un proceso para definir una ontología mediante clases, jerarquías, propiedades, restricciones e instancias. OWL 2 formaliza clases, propiedades, individuos y valores para representar conocimiento con semántica definida (W3C, 2012). Por su parte, Neo4j documenta Cypher como lenguaje declarativo para consultar nodos, relaciones y patrones. Se combinan ambos enfoques: OWL formaliza el vocabulario y sus restricciones; el grafo ofrece consultas operativas sobre relaciones y permite construir recomendaciones candidatas por caminos de interés.

5. Metodología de Ingeniería Web

5.1 Selección: UWE

Se propone UWE (UML-based Web Engineering) como metodología de modelado, apropiada para expresar de manera separada los requisitos de una aplicación web, el modelo conceptual, la navegación y la presentación. Esta elección se conecta con la necesidad del curso de entregar modelos navegacionales, conceptuales y prototipo de interfaz. La documentación consultada sobre métodos de ingeniería web dirigidos por modelos muestra la relevancia de representar la estructura y navegación antes de generar o implementar las interfaces (Hincapié Londoño & Duitama, 2012).

5.2 Fases aplicadas al proyecto

Fase	Actividades aplicadas	Productos / relación con usuarios
1. Requisitos	Formular el problema, identificar actores, redactar historias, priorizar funcionalidades y reglas.	Ficha del proyecto, requisitos RF/RNF, reglas de negocio y preguntas de competencia.
2. Modelo conceptual	Identificar estudiantes, perfiles, competencias, temas, recursos, cursos, recomendaciones y retroalimentación.	Modelo conceptual y vocabulario que el equipo utiliza de forma consistente.
3. Modelo navegacional	Definir rutas desde panel a perfil, recomendaciones, detalle, catálogo y asesoría.	Mapa de navegación centrado en tareas, con rutas cortas y control del usuario.
4. Modelo de presentación	Diseñar jerarquía visual, componentes, etiquetas, filtros, tarjetas y estados.	Prototipo navegable HTML/CSS/JS y evaluación heurística inicial.
5. Modelo de proceso / lógica	Especificar servicios REST y la lógica transparente que genera candidatos y explicación.	OpenAPI, ejemplo SOAP/WSDL, pseudológica de recomendación y consultas Cypher.
6. Evaluación y refinamiento	Revisar trazabilidad, coherencia de restricciones, usabilidad, privacidad y funcionamiento de scripts.	Lista de pruebas, defectos, evidencia ejecutable y mejoras priorizadas.

5.3 Plan de iteración propuesto

Iteración	Objetivo	Validación de salida
1	Definir tema, problema, actores y requisitos.	Aprobación docente y RF priorizados.
2	Completar modelos conceptual/navegacional y arquitectura.	Relaciones coherentes entre modelos y funcionalidades.
3	Crear prototipo y revisar usabilidad.	Flujos principales navegables y lista de mejoras.

Iteración	Objetivo	Validación de salida
4	Construir y revisar ontología OWL.	Archivo abre en Protégé, vocabulario consistente y razonador sin inconsistencias reportadas.
5	Crear semilla y consultas de Neo4j.	Consultas devuelven resultados esperados con datos sintéticos.
6	Integrar, documentar, preparar evidencias y sustentar.	Documento, archivos, repositorio y presentación coherentes.

5.4 Ventajas y limitaciones

Ventajas: UWE orienta la construcción de modelos específicos para aplicaciones web, permite discutir navegación y presentación por separado, y facilita la trazabilidad. Limitaciones: una metodología de modelado no garantiza por sí sola la calidad de las recomendaciones ni resuelve la gobernanza de datos; la ontología y el grafo requieren mantenimiento, y las reglas iniciales deberán probarse con personas expertas y datos de evaluación autorizados.

6. Arquitectura web

6.1 Estilo seleccionado

Arquitectura lógica por capas con interfaz web responsive, API REST, servicios de aplicación, motor de recomendación explicable, persistencia relacional para perfiles/metadatos, ontología OWL para semántica formal y Neo4j para consultas sobre conexiones. El ejemplo SOAP/WSDL se aísla mediante un adaptador de interoperabilidad; no se propone duplicar toda la API en SOAP.

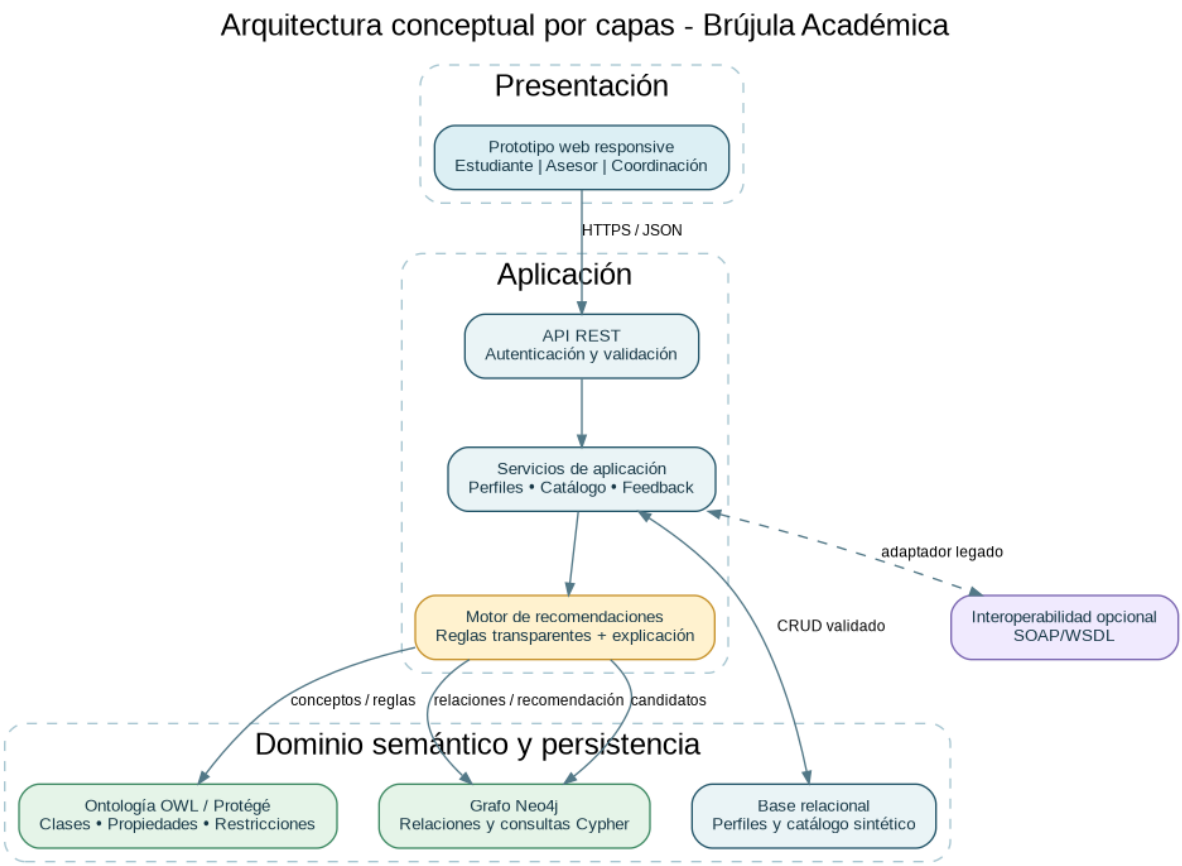


Figura 1. Arquitectura conceptual. Es un diseño lógico, no un despliegue ya construido.

6.2 Responsabilidad de los componentes

Componente	Responsabilidad	Justificación
Interfaz web	Presenta panel, perfil, catálogo, recomendaciones y asesoría; valida entradas básicas y comunica estados.	Mejora usabilidad; mantiene separada la presentación.
API REST	Expone recursos, valida contratos, autentica/autoriza en una futura implementación y normaliza errores.	Interfaz uniforme y desacoplamiento.
Servicios de aplicación	Orquestan perfiles, catálogo, retroalimentación y resúmenes.	Evita mezclar reglas del dominio con componentes visuales.
Motor de recomendación	Calcula afinidad por tema, competencia y restricciones; devuelve explicación y puntaje relativo.	Resultados comprensibles y sujetos a ajuste.
Persistencia relacional	Guarda datos operativos como perfiles, catálogo, feedback y permisos.	Integridad transaccional para registros.
Ontología OWL	Define clases, propiedades, jerarquías y restricciones del dominio.	Lenguaje compartido e inferencias semánticas.

Componente	Responsabilidad	Justificación
Grafo Neo4j	Representa conexiones entre personas sintéticas, intereses, competencias, recursos y recomendaciones.	Consultas por patrones y relaciones no triviales.
Adaptador SOAP/WSDL	Transforma solicitudes de un sistema legado a la operación interna y serializa la respuesta SOAP.	Interoperabilidad sin contaminar el resto del diseño.

6.3 Atributos de calidad

Atributo	Decisión de diseño	Cómo se verificaría
Mantenibilidad	Capas, nombres consistentes, contrato OpenAPI y separación ontología/grafo.	Revisión de dependencias y modificación aislada de una regla.
Seguridad	HTTPS, autorización por rol, validación de entrada y mínimos privilegios.	Pruebas de acceso denegado, validación, registros y revisión de configuración.
Escalabilidad	Servicios stateless y acceso desacoplado a almacenamiento/semántica.	Pruebas de carga y perfilado; aún no ejecutados.
Usabilidad	Flujos por tareas, estados claros, filtros y explicaciones.	Heurísticas y pruebas de tarea con participantes; pendientes.
Interoperabilidad	API REST y contrato SOAP/WSDL para un único caso.	Validación de esquema y prueba de solicitud/respuesta de extremo a extremo.

7. Modelado conceptual, casos de uso y navegación

7.1 Actores y casos de uso

Actor	Casos de uso principales
Estudiante	Actualizar perfil; elegir intereses; consultar competencias; explorar catálogo; ver recomendaciones explicadas; guardar/ocultar/valorar recursos.
Asesor académico	Consultar resumen agregado; identificar temas frecuentes; sugerir recursos o talleres.
Coordinador/curador	Crear/editar recurso; asociar recursos a temas/competencias; validar nivel y horas.
Administrador técnico	Gestionar roles, configuración y auditoría en una futura implementación.

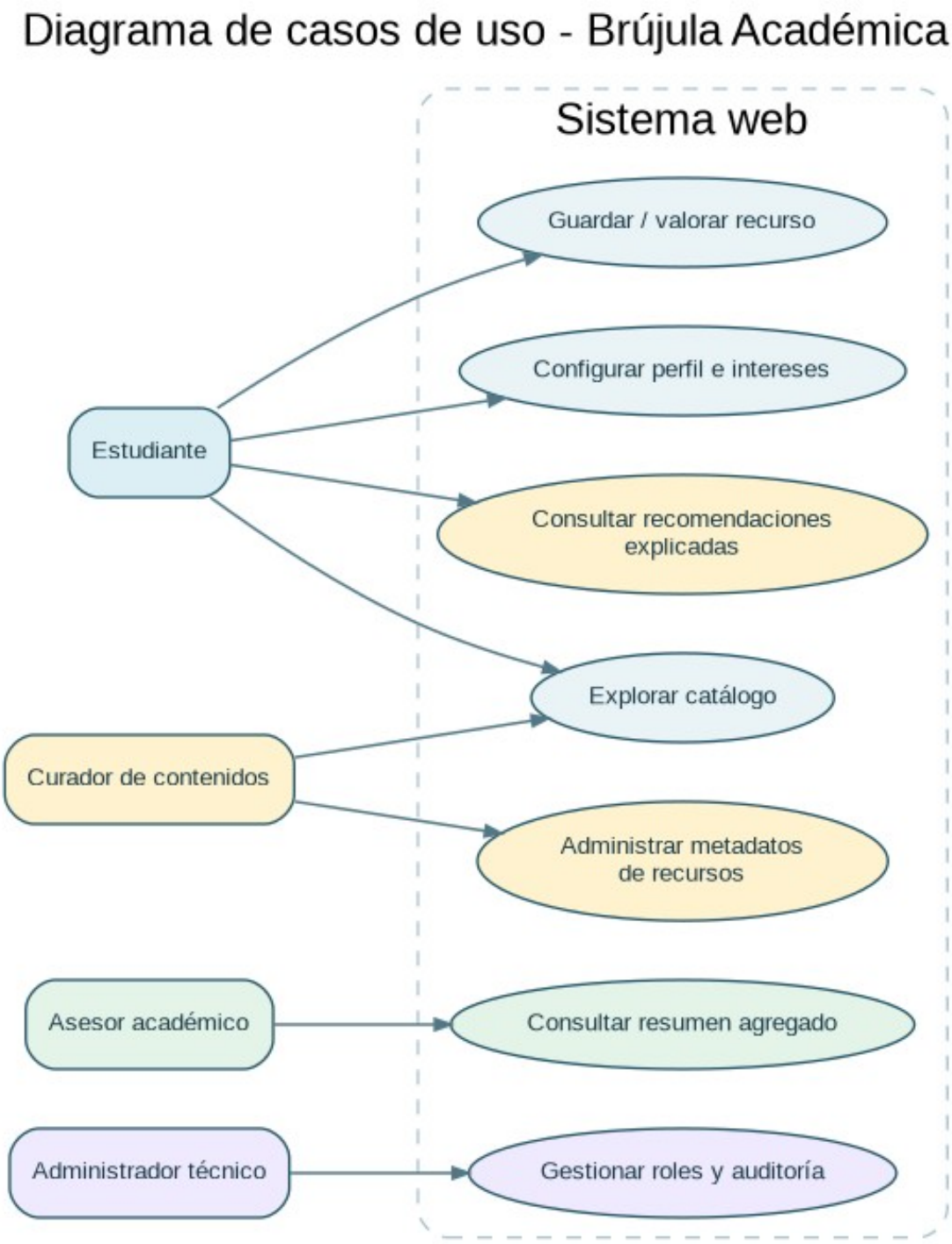


Figura 2. Diagrama de casos de uso; los permisos de producción requieren validación adicional.

7.2 Modelo conceptual del dominio

Las entidades se conectan mediante relaciones de dominio, no por mera similitud de etiquetas. Por ejemplo, un recurso cubre uno o varios temas y fortalece una o varias competencias; un curso requiere competencias; una recomendación se genera para un estudiante y sugiere recursos a partir de uno o más criterios. La figura refleja las relaciones principales; el archivo OWL las formaliza con rangos y dominios.

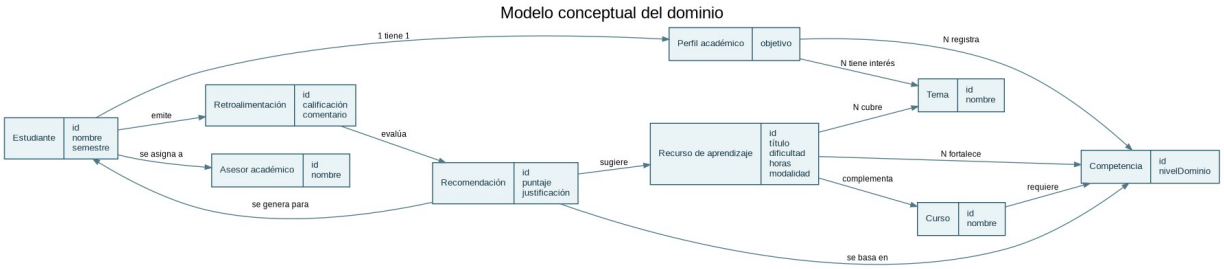


Figura 2. Modelo conceptual de Brújula Académica.

7.3 Modelo navegacional

La navegación parte de un panel personal y permite llegar al perfil, la autoevaluación, las recomendaciones, el catálogo y el detalle de recurso. El retorno al conjunto de sugerencias permite corregir filtros o descubrir opciones alternativas. La vista del asesor se separa por rol y muestra información agregada.

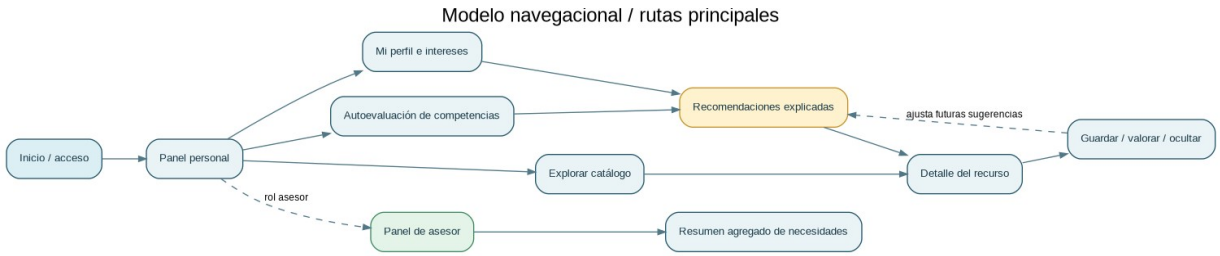


Figura 3. Modelo navegacional con la ruta del estudiante y el acceso por rol del asesor.

7.4 Trazabilidad de casos de uso

Caso	Entrada	Resultado esperado	Requisito relacionado
CU-01 Configurar perfil	Objetivo, intereses, competencias autodeclaradas.	Perfil actualizado y visible para el usuario.	RF-01
CU-02 Explorar catálogo	Texto y filtros por tema/nivel/modalidad.	Recursos compatibles con los filtros.	RF-02, RF-08
CU-03 Solicitar recomendaciones	Id. de estudiante y filtros opcionales.	Lista ordenada con puntaje relativo y razones.	RF-03, RF-04
CU-04 Dar retroalimentación	Id. de recomendación, valoración y acción.	Registro de feedback y confirmación visual.	RF-05
CU-05 Consultar patrón agregado	Id. de asesor autorizado.	Resumen por tema/competencia sin ranking individual.	RF-07
CU-06 Explorar una inferencia	Interés y recursos conectados por el grafo.	Candidatos no registrados directamente como recomendación.	RF-06

8. Prototipo y usabilidad

8.1 Interfaces principales

- Panel del estudiante: saludo, indicadores de ejemplo y recomendaciones explicadas como tarjetas.
- Mi perfil: objetivo y temas de interés editables en modo demostración.
- Catálogo: tabla filtrable por tema y búsqueda textual.
- Vista de asesoría: cifras y patrones agregados ficticios, sin ranking ni etiquetas individuales.
- Acciones de recurso: consultar detalle conceptual, guardar u ocultar, con retroalimentación visual.

Cómo abrir el prototipo

Descomprimir el paquete y abrir prototipo/index.html en un navegador moderno. No necesita servidor, cuenta ni conexión a internet; todas las acciones operan localmente y no persisten al cerrar la página.

8.2 Evaluación heurística inicial

La siguiente revisión es una evaluación por inspección del diseño propuesto inspirada en los principios de Nielsen (1994/2024). No representa un estudio con participantes ni afirma que el prototipo haya sido validado empíricamente. En una iteración posterior se debe comprobar cada mejora mediante tareas con estudiantes y asesores.

Heurística	Aplicación / riesgo observado	Mejora propuesta o criterio
Visibilidad del estado	Guardar u ocultar cambia la vista y puede generar dudas sobre el resultado.	Mostrar mensaje de confirmación con región aria-live; indicar filtros activos.
Correspondencia con el mundo real	Términos semánticos como “competencia” pueden ser abstractos.	Usar ejemplos cotidianos y etiquetas como “qué puedes practicar” junto al término formal.
Control y libertad	Una recomendación ocultada no debería considerarse decisión irreversible.	Ofrecer acción para restablecer filtro y controlar preferencias del usuario.
Consistencia y estándares	Botones, filtros y etiquetas deben conservar estilos y verbos.	Usar el mismo estilo para filtros y acciones con iconos etiquetados.
Prevención de errores	Los valores de valoración o duración podrían ser inválidos en un formulario futuro.	Validar rangos, mostrar límites antes de enviar y evitar acciones destructivas sin confirmación.
Reconocimiento vs. recuerdo	El estudiante no debería memorizar por qué se le sugirió un recurso.	Mostrar la explicación dentro de cada tarjeta y repetirla en la ficha de detalle.
Estética minimalista	Demasiadas métricas podrían competir con la acción principal.	Mantener tres indicadores de apoyo y priorizar las tarjetas recomendadas.
Ayuda y recuperación	El prototipo es autoexplicativo, pero no incluye centro de ayuda completo.	Agregar glosario, FAQ y ruta de soporte antes de un despliegue real.

8.3 Pruebas de usabilidad futuras

Tarea	Participante previsto	Métrica / señal
Encontrar un recurso básico de programación de menos de 5 horas.	Estudiante	Éxito de tarea, tiempo y errores de filtro.
Explicar por qué se recomienda un recurso.	Estudiante	Comprensión de la justificación, no solo clic.
Cambiar intereses y verificar si la lista cambia.	Estudiante	Tasa de finalización y percepción de control.
Identificar un tema agregado que requiera un taller.	Asesor	Tiempo, exactitud y claridad del resumen.
Asociar un recurso con una competencia correcta.	Curador	Errores de metadatos y completitud del formulario.

9. Diseño de servicios web REST y SOAP/WSDL

9.1 Convenciones REST

Base conceptual: /v1. Los recursos se identifican con URI, los métodos HTTP indican la semántica de la operación, los cuerpos se intercambian como JSON y los errores incluyen un código HTTP y un mensaje comprensible. La especificación completa de referencia está en api/openapi.yaml. Los dominios example.org y la dirección del servicio SOAP son marcadores, no endpoints reales.

Método y ruta	Propósito	Entrada principal	Salida esperada	Usuario/componente
GET /students/{id}/profile	Consultar perfil	studentId en ruta.	Objetivo, intereses y competencias declaradas.	Estudiante / interfaz
PUT /students/{id}/profile	Actualizar perfil	Objetivo, lista de intereses, competencias.	Perfil actualizado o error 400.	Estudiante / servicio de perfil
GET /students/{id}/recommendations	Generar/listar sugerencias	studentId; limit; modalidad opcional.	Lista de recursos, score y explicación.	Estudiante / motor
GET /resources	Explorar catálogo	topic, level, modality, query opcionales.	Recursos filtrados con metadatos.	Estudiante / curador
GET /resources/{resourceId}	Consultar detalle	resourceId.	Descripción, temas, competencias, duración.	Estudiante / interfaz
POST /recommendations/{id}/feedback	Registrar opinión	rating 1-5, comment opcional y acción.	201 Created o 400 de validación.	Estudiante / servicio de feedback
GET /advisors/{id}/summary	Ver resumen agregado	advisorId validado por rol.	Conteos/patrones agregados por tema.	Asesor / servicio analítico
POST /admin/resources	Crear recurso de catálogo	Título, temas, competencias, nivel, horas, modalidad.	201 Created con id asignado.	Curador autorizado

9.2 Ejemplo de solicitud y respuesta

```
GET /v1/students/STU-001/recommendations?limit=5
Accept: application/json


HTTP/1.1 200 OK
Content-Type: application/json
[
 {
  "id": "REC-001",
  "resourceId": "RES-PY-01",
  "title": "Python desde cero",
  "score": 0.92,
  "estimatedHours": 4,
  "explanation": [
   "Coincide con el interés en programación",
   "Aborda la competencia Python que se desea fortalecer"
  ]
 }
]
```

9.3 Códigos de error principales

HTTP	Uso esperado
400 Bad Request	Parámetros inválidos o valoración por fuera de 1 a 5.
401 Unauthorized	No existe sesión válida cuando se implemente autenticación.
403 Forbidden	Rol sin permisos o intento de consultar datos ajenos.
404 Not Found	Perfil, recurso o recomendación inexistente.
409 Conflict	Conflicto de actualización o duplicidad de un recurso único.
500 Internal Server Error	Fallo inesperado; no incluir stack traces ni datos sensibles en la respuesta.

9.4 Representación SOAP/WSDL

Como ejemplo de interoperabilidad con un sistema académico legado se define la operación RecomendarRecursos. La solicitud contiene studentId y un límite opcional; la respuesta devuelve una colección de recursos con identificador, título, puntaje relativo y explicación. WSDL describe mensajes, tipos, operación, binding y dirección lógica del servicio. En la solución final propuesta, un adaptador SOAP traduciría esta petición a la función de recomendación interna y regresaría el resultado como XML.

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:ba="https://example.org/brujula-academica/soap">
 <soapenv:Header/>
 <soapenv:Body>
  <ba:RecomendarRecursosRequest>
   <ba:studentId>STU-001</ba:studentId><ba:limit>5</ba:limit>
  </ba:RecomendarRecursosRequest>
 </soapenv:Body>
</soapenv:Envelope>
```

El WSDL y el ejemplo XML se entregan como artefactos de diseño. Antes de usarlos en integración real deben validarse en un motor SOAP y definirse seguridad, transporte, autenticación, versionado y manejo de fallos.

10. Ontología del dominio en OWL

10.1 Propósito y alcance

La ontología Brújula Académica ofrece un vocabulario explícito para describir estudiantes, perfiles académicos, competencias, recursos, cursos, temas, recomendaciones, retroalimentación y asesores. El archivo ontologia/BrujulaAcademica.owl se serializa en RDF/XML y puede abrirse en Protégé. La ontología contiene más que el mínimo requerido por la guía: 15 clases, 14 propiedades de objeto, 10 propiedades de datos, restricciones e individuos sintéticos.

10.2 Clases

Clase	Significado / uso
Student	Persona estudiante del modelo de demostración.
AcademicProfile	Objetivos, intereses y competencias declaradas.
Competency	Conocimiento o habilidad que puede fortalecerse.
ProgrammingCompetency	Subclase de Competency para programación y lógica.
DataCompetency	Subclase de Competency para datos y consultas.
APICompetency	Subclase de Competency para diseño de servicios.
LearningResource	Guía, laboratorio, actividad o recurso de aprendizaje.
Course	Curso curricular al que puede complementar un recurso.
Topic	Tema académico como programación, datos o desarrollo web.
Recommendation	Asociación de recursos sugeridos a un estudiante.
Feedback	Valoración o acción realizada por un estudiante.
Advisor	Asesor académico.
ProgrammingResource	Clase definida por recurso que fortalece alguna competencia de programación.
BasicResource	Clase definida por nivel de dificultad “Básico”.
ResourceForData	Clase definida por recurso que fortalece alguna competencia de datos.

10.3 Propiedades de objeto (dominio y rango)

Propiedad	Dominio	Rango	Lectura funcional
hasProfile	Student	AcademicProfile	tiene perfil

Propiedad	Dominio	Rango	Lectura funcional
hasInterest	AcademicProfile	Topic	tiene interés
hasCompetency	AcademicProfile	Competency	registra competencia
addressesCompetency	LearningResource	Competency	fortalece competencia
coversTopic	LearningResource	Topic	cubre tema
requiresCompetency	Course	Competency	requiere competencia
recommendsResource	Recommendation	LearningResource	recomienda recurso
generatedFor	Recommendation	Student	se genera para
basedOnCompetency	Recommendation	Competency	se basa en competencia
assignedAdvisor	Student	Advisor	tiene asesor
givesFeedback	Student	Feedback	emite retroalimentación
aboutRecommendation	Feedback	Recommendation	evalúa recomendación
supplementsCourse	LearningResource	Course	complementa curso
relatedTopic	Topic	Topic	tema relacionado con

10.4 Propiedades de datos

Propiedad	Dominio	Tipo	Uso
studentName	Student	xsd:string	Nombre sintético
semester	Student	xsd:integer	Semestre representado
profileGoal	AcademicProfile	xsd:string	Objetivo
proficiencyLevel	Competency	xsd:decimal	Dominio estimado entre 0 y 1
resourceTitle	LearningResource	xsd:string	Título del recurso
difficultyLevel	LearningResource	xsd:string	Nivel de dificultad
estimatedHours	LearningResource	xsd:decimal	Duración estimada
modality	LearningResource	xsd:string	Virtual, presencial o mixta
recommendationScore	Recommendation	xsd:decimal	Afinidad relativa
feedbackRating	Feedback	xsd:integer	Valoración entera

10.5 Restricciones

N.º	Formalización OWL	Intención de dominio
1	Student SubClassOf hasProfile exactly 1 AcademicProfile	Cada estudiante tiene un perfil académico en el modelo conceptual.

N.º	Formalización OWL	Intención de dominio
2	Recommendation SubClassOf generatedFor exactly 1 Student	Cada recomendación se refiere a un estudiante.
3	Recommendation SubClassOf recommendsResource some LearningResource	Una recomendación sugiere al menos un recurso.

OWL utiliza una hipótesis de mundo abierto: la ausencia de un dato no equivale automáticamente a que sea falso. Las restricciones describen condiciones semánticas, pero la calidad de los datos debe comprobarse por separado; por ejemplo, un formulario REST debe validar horas no negativas y puntuaciones entre cero y uno.

10.6 Individuos de demostración

Tipo	Individuos ejemplo
Estudiantes/perfiles	studentAna / profileAna; studentLuis / profileLuis.
Asesores	advisorMaria; advisorCarlos.
Competencias	competencyPython; competencySQL; competencyREST; competencyLogic.
Temas	topicProgramming; topicData; topicWeb; topicAlgorithms.
Recursos/cursos	resourcePythonBasics; resourceSQLLab; resourceRESTGuide; resourceAlgoPractice; courseAlgorithms.
Recomendaciones/feedback	recommendationAnaPython; recommendationLuisSQL; feedbackAnaPython.

10.7 Preguntas de competencia

1. PC-01. ¿Qué recursos cubren un tema que interesa a un estudiante y fortalecen una competencia de su perfil?
2. PC-02. ¿Qué recursos de nivel básico abordan competencias de programación que el estudiante desea practicar?
3. PC-03. ¿Qué recomendaciones se han asociado a un estudiante y qué competencias se usaron para justificarlas?
4. PC-04. ¿Qué recursos complementan un curso y cuáles son las competencias que dicho curso requiere?

10.8 Inferencias esperadas y verificación

La definición de clases usa equivalencias lógicas para permitir clasificación automática. Las siguientes son inferencias esperadas al ejecutar un razonador compatible con OWL 2, por ejemplo HermiT, en Protégé. No se marcan como capturas ejecutadas: deben validarse y documentarse en la aplicación local antes de la entrega final.

Inferencia esperada	Regla que la permite	Qué comprobar en Protégé
resourcePythonBasics se clasifica también como ProgrammingResource.	ProgrammingResource equivale a LearningResource y a un recurso que addressesCompetency algún ProgrammingCompetency; resourcePythonBasics fortalece competencyPython, que es ProgrammingCompetency.	Ejecutar HermiT y revisar los tipos inferidos del individuo.
resourcePythonBasics se clasifica como BasicResource.	BasicResource equivale a LearningResource con difficultyLevel “Básico”; el individuo tiene esa propiedad.	Revisar la pestaña Types / Inferred types.
resourceSQLLab se clasifica como ResourceForData.	ResourceForData equivale a LearningResource que fortalece algún DataCompetency; competencySQL pertenece a esa subclase.	Ejecutar el razonador y revisar que el individuo quede clasificado.

10.9 Consistencia y límites

El archivo declara clases, dominios/rangos, restricciones y hechos sintéticos coherentes con el modelo. La comprobación definitiva consiste en abrir el OWL, ejecutar HermiT, verificar el estado de consistencia, revisar clases/individuos inferidos y guardar capturas. La inferencia OWL clasifica conocimiento descrito; no sustituye el motor de recomendaciones ni demuestra eficacia pedagógica.

11. Grafo de conocimiento en Neo4j

11.1 Modelo del grafo

El grafo opera como representación consultable de conexiones entre estudiantes sintéticos, perfiles, asesores, temas, competencias, recursos, cursos, recomendaciones y retroalimentación. El archivo neo4j/brujula_academica.cypher crea restricciones, carga la semilla y propone cuatro consultas principales más dos consultas de verificación. El modelo contiene más de ocho nodos y más de diez relaciones.

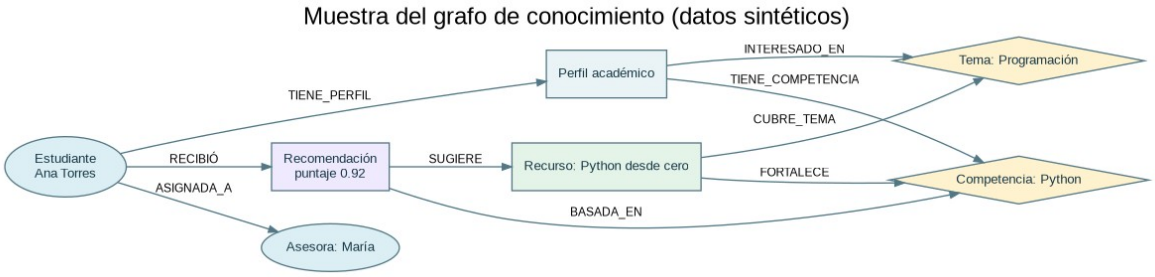


Figura 4. Fragmento ilustrativo de la red de conocimiento.

11.2 Etiquetas y relaciones

Nodos	Relaciones principales
Student, AcademicProfile, Advisor, Topic, Competency, Resource, Course, Recommendation, Feedback.	HAS_PROFILE, ASSIGNED_TO, INTERESTED_IN, HAS_COMPETENCY, COVERS_TOPIC, ADDRESSES_COMPETENCY, REQUIRES_COMPETENCY, SUPPLEMENTS, RECEIVED, SUGGESTS, BASED_ON, GAVE_FEEDBACK, ABOUT.

11.3 Consultas y necesidad que atienden

Consulta	Necesidad	Resultado
1. Recomendaciones existentes	Explicar qué se le ha recomendado a un estudiante y por qué.	Recurso, nivel, horas, puntaje, explicación y competencias.
2. Recomendación inferida	Encontrar un recurso que comparte un tema de interés y no está guardado como recomendación directa.	Candidatos calculados por patrón del grafo con puntaje relativo y razón.
3. Resumen por tema	Apoyar planeación de talleres y recursos de asesoría.	Conteos agregados e indicador de dominio promedio.
4. Ruta recurso-curso	Comprender cómo un recurso puede preparar para un curso que requiere una competencia.	Curso, recurso preparatorio y competencia requerida.

11.4 Consulta con recomendación no registrada directamente

La consulta 2 sigue una ruta estudiante → perfil → tema de interés ← recurso que cubre el tema, comprueba si hay una competencia de bajo dominio abordada por el recurso y excluye los recursos que ya se habían vinculado a una recomendación. Así puede devolver un candidato que no estaba guardado como recomendación. En el ejemplo de Ana, “Retos de lógica y algoritmos” puede surgir porque coincide con el tema “Algoritmos”, mientras que la recomendación registrada inicialmente es “Python desde cero”. Es una inferencia por patrón de datos del grafo, no una nueva clase inferida por OWL.

```
MATCH (s:Student {id:$studentId})-[:HAS_PROFILE]->(p:AcademicProfile)-[:INTERESTED_IN]->(t:Topic)<-[:COVERS_TOPIC]-(r:Resource)
WHERE NOT EXISTS {
  MATCH (s)-[:RECEIVED]->(:Recommendation)-[:SUGGESTS]->(r)
}
RETURN s.name, r.title, collect(DISTINCT t.name) AS temas
LIMIT 5;
```

11.5 Ejecución y evidencias requeridas

1. Abrir Neo4j Desktop o una instancia de Neo4j 5.x con una base de prueba.
2. Abrir neo4j/brujula_academica.cypher y ejecutar primero el bloque de restricciones y semilla.
3. Ejecutar de forma individual las consultas, proporcionando parámetros como studentId = STU-001 y advisorId = ADV-001.
4. Guardar captura de nodos/relaciones en el navegador de Neo4j, resultados de cada una de las tres consultas obligatorias y la consulta inferida.

5. Anotar la versión de Neo4j utilizada y cualquier ajuste de compatibilidad. Las capturas aún deben ser tomadas por el equipo.

12. Integración, trazabilidad y evaluación

12.1 Integración de componentes

Pregunta de integración	Respuesta en el diseño
¿Cómo se relaciona el problema con las funcionalidades?	La dificultad de seleccionar recursos origina perfiles, filtros, recomendaciones explicadas y retroalimentación.
¿Cómo se conecta el modelo conceptual con la navegación?	Las entidades se presentan como tareas: el perfil configura intereses; las recomendaciones y el catálogo muestran recursos; la ficha explica temas y competencias.
¿Cómo representa la ontología el dominio?	Formaliza clases, relaciones, dominios/rangos y restricciones para compartir un vocabulario y clasificar individuos.
¿Cómo usan los servicios los conceptos?	Los contratos REST intercambian identificadores y atributos de perfil, tema, competencia, recurso y recomendación.
¿Cómo aporta conocimiento el grafo?	La consulta por caminos descubre candidatos vinculados a intereses, recursos y competencias aun si no existe una recomendación guardada.
¿Cómo soporta la arquitectura la solución?	La interfaz consume la API; los servicios coordinan reglas y almacenamiento; OWL y Neo4j cubren necesidades semánticas diferentes.

12.2 Matriz de trazabilidad

Necesidad	Funcionalidad	Modelo/ontología	Servicio/consulta	Criterio de aceptación
Encontrar material relevante	RF-02 catálogo	Topic, LearningResource; coversTopic	GET /resources; consulta de candidatos	Filtrar por tema devuelve solo recursos compatibles.
Fortalecer una competencia	RF-03 recomendación	Competency; addressesCompetency	GET /students/{id}/recommendations; Cypher 2	Cada sugerencia muestra competencia y explicación.
Comprender motivo	RF-04 explicación	Recommendation; basedOnCompetency	JSON explanation; Cypher 1	Cada recurso recomendado tiene al menos una razón.
Dar opinión	RF-05 feedback	Feedback; aboutRecommendation	POST /recommendations/{id}/feedback	Valida puntuación 1-5 y confirma el resultado.
Planear apoyo grupal	RF-07 resumen asesor	Advisor, Topic, Competency	GET /advisors/{id}/summary; Cypher 3	Respuesta agregada y controlada por rol.

Necesidad	Funcionalidad	Modelo/ontología	Servicio/consulta	Criterio de aceptación
Consultar conocimiento semántico	RF-06 dominio	OWL, restricciones y clases definidas	Protégé/HermiT; Cypher 2 y 4	Consistencia y consultas verificadas con evidencias.

12.3 Evaluación cuantitativa futura

Métrica	Definición operacional	Meta inicial sugerida
Precisión@5	Proporción de los cinco recursos más altos que evaluadores académicos consideran relevantes para el objetivo sintético.	Medir con conjunto etiquetado por una persona experta; no fijar éxito hasta conocer línea base.
Cobertura de catálogo	Porcentaje de recursos elegibles que pueden ser sugeridos a al menos un perfil de evaluación.	Detectar recursos nunca recomendados.
Diversidad temática	Número de temas distintos entre las recomendaciones mostradas.	Evitar que todas las sugerencias sean casi iguales.
Tasa de explicación	Porcentaje de recomendaciones que incluyen al menos una razón comprensible.	100% en el contrato funcional.
Éxito de tarea de usabilidad	Porcentaje de participantes que completan tareas previstas sin ayuda.	Definir tras prueba piloto y reportar muestra/limitaciones.
Latencia API	Tiempo de respuesta bajo carga acordada.	P95 menor a 2 s como objetivo de diseño; debe medirse.

13. Seguridad, privacidad y uso responsable

- Minimización de datos: el modelo solo requiere intereses, objetivos y competencias autodeclaradas necesarias para personalizar sugerencias. El proyecto académico emplea únicamente datos sintéticos.
- Consentimiento y transparencia: explicar qué información se usa, con qué propósito y cómo puede cambiarse u ocultarse.
- Control de acceso: un estudiante solo consulta y modifica su propio perfil; el asesor ve únicamente información agregada autorizada; el curador administra metadatos de recursos.
- No discriminación: no usar recomendaciones para decidir matrícula, sanciones, becas, permanencia o capacidad académica. Evitar convertir un nivel autodeclarado en una etiqueta permanente.
- Explicaciones y agencia: mostrar los criterios y permitir rechazar una sugerencia; las recomendaciones deben ser apoyo y no instrucción obligatoria.
- Calidad del catálogo: mantener autoría, vigencia, nivel, duración, licencias y revisión pedagógica de recursos antes de publicarlos.
- Privacidad en agregados: en producción, definir grupos mínimos, retención, anonimización y controles para reducir el riesgo de reidentificación.
- Accesibilidad: realizar pruebas técnicas y pruebas con usuarios con diversas necesidades; no dar por garantizada la accesibilidad solo por usar diseño responsive.

Limitación ética clave
Un sistema de recomendación puede reforzar sesgos si el catálogo es limitado o las preferencias están incompletas. Para mitigarlo se deben revisar cobertura, diversidad, explicabilidad y control por parte del estudiante, con supervisión humana.

14. Plan de trabajo y criterios de aceptación

14.1 Roles sugeridos para un equipo de hasta cuatro personas

Rol sugerido	Responsabilidad principal	Entregables que lidera
Integrante A - requisitos y documentación	Problema, fuentes, alcance, trazabilidad y coordinación del informe.	Documento técnico, matriz de requisitos, bibliografía.
Integrante B - modelado web y prototipo	Arquitectura, navegación, interfaces y heurísticas.	Diagramas, HTML navegable, evaluación UX.
Integrante C - ontología	Clases, propiedades, restricciones, individuos y razonamiento.	OWL, captura de Protégé, explicación de inferencias.
Integrante D - grafo e integración	Modelo Neo4j, consultas, servicios y preparación de demo.	Cypher, capturas de Neo4j, OpenAPI/WSDL, presentación.

Los roles son una propuesta de reparto y deben adaptarse a los integrantes reales. Todos deben comprender el dominio, la arquitectura, la ontología y las consultas porque la guía del curso exige participación de todos durante la sustentación.

14.2 Criterios de aceptación de los artefactos

Artefacto	Criterio de aceptación	Estado en este paquete
Documento técnico	Problema, usuarios, objetivos, alcance, al menos cinco fuentes, arquitectura, metodología y trazabilidad.	Preparado para revisión del equipo y del docente.
Diagramas	Arquitectura, modelo conceptual, navegación y fragmento de grafo alineados.	Incluidos como PNG, DOT y Mermaid.
Prototipo	Abre localmente y ofrece navegación y filtros principales.	Archivo HTML entregado; probar en navegador antes de presentar.
OWL	Clases, propiedades, dominios/rangos, restricciones, individuos y preguntas de competencia.	Archivo generado; ejecutar Protégé/HermiT y capturar evidencias.
Cypher	Semilla con más de ocho nodos, relaciones suficientes y tres consultas funcionales.	Script entregado; ejecutar en Neo4j y guardar resultados.
REST / SOAP	Rutas principales documentadas y una operación SOAP/WSDL de referencia.	OpenAPI, WSDL y XML de solicitud entregados; validar sintaxis y motor local.

Artefacto	Criterio de aceptación	Estado en este paquete
Presentación	Explicación del problema, modelos, arquitectura, ontología, grafo, recomendaciones y límites.	Diapositivas incluidas.
Repositorio/historial	Cambios y contribuciones individuales trazables.	El equipo debe crear/enlazar su repositorio y registrar el trabajo real.

14.3 Lista de verificación antes de entregar

- ☐ Confirmar la aprobación docente del tema antes de tratar esta versión como proyecto autorizado.
- ☐ Sustituir [Integrantes] y [Docente], revisar portada, ortografía, alcance y pertinencia de cada apartado.
- ☐ Abrir el HTML en un navegador y recorrer panel, perfil, catálogo, filtros, guardar y ocultar.
- ☐ Abrir el OWL en Protégé, ejecutar un razonador (por ejemplo HermiT), verificar consistencia y guardar capturas auténticas.
- ☐ Ejecutar el script Cypher en una base de prueba, verificar conteos, ejecutar las tres consultas requeridas y guardar capturas.
- ☐ Validar OpenAPI y WSDL con una herramienta apropiada y documentar cualquier ajuste.
- ☐ Crear un repositorio Git, añadir artefactos e historial real de aportes y compartir el enlace de solo lectura o acceso requerido.
- ☐ Ajustar las diapositivas a la distribución real del equipo y practicar una sustentación de máximo 20 minutos.

15. Conclusiones y trabajo futuro

Brújula Académica atiende una necesidad concreta de orientación de recursos mediante un diseño web que relaciona intereses, competencias y contenidos con explicaciones visibles. El valor académico está en la integración coherente: el modelo conceptual describe entidades, la navegación organiza tareas, la API expone recursos, la ontología formaliza el vocabulario y Neo4j permite descubrir rutas de conexión para recomendar candidatos no registrados directamente.

La propuesta cumple los mínimos conceptuales solicitados en la guía del curso y los amplía con contratos API, reglas de privacidad, criterios de aceptación y un prototipo funcional de demostración. Sin embargo, el tema está pendiente de aprobación docente y las pruebas externas no deben darse por ejecutadas hasta contar con evidencias auténticas de Protégé y Neo4j.

Trabajo futuro

- Validar el problema con entrevistas o pruebas de tarea a estudiantes, asesores y curadores.
- Comparar la recomendación basada en contenido y grafo con una línea base de búsqueda por palabras clave.
- Crear un conjunto sintético etiquetado por expertos y calcular métricas de relevancia, diversidad y cobertura.
- Integrar un catálogo académico autorizado y evaluar consentimiento, minimización de datos y permisos.
- Validar inferencias OWL, compatibilidad Cypher, seguridad y accesibilidad mediante pruebas reproducibles.

Referencias

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Hincapié Londoño, J. A., & Duitama, J. F. (2012). Model-driven web engineering methods: A literature review. Revista Facultad de Ingeniería Universidad de Antioquia, (63), 69–81. https://doi.org/10.17533/udea.redin.12487

International Organization for Standardization. (2019). Ergonomics of human-system interaction—Part 210: Human-centred design for interactive systems (ISO Standard No. 9241-210:2019). https://www.iso.org/standard/77520.html

Murugesan, S., Deshpande, Y., Hansen, S., & Ginige, A. (2001). Web engineering: A new discipline for development of web-based systems. In S. Murugesan & Y. Deshpande (Eds.), Web Engineering (pp. 3–13). Springer. https://doi.org/10.1007/3-540-45144-7_2

Nielsen, J. (2024, January 30). 10 usability heuristics for user interface design. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Noy, N. F., & McGuinness, D. L. (2001). Ontology development 101: A guide to creating your first ontology (Technical Report KSL-01-05). Stanford University. https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html

Silva, F. L. da, Slodkowski, B. K., Silva, K. K. A. da, & Cazella, S. C. (2023). A systematic literature review on educational recommender systems for teaching and learning: Research trends, limitations and opportunities. Education and Information Technologies, 28, 3289–3328. https://doi.org/10.1007/s10639-022-11341-9

World Wide Web Consortium. (2012). OWL 2 web ontology language primer (Second Edition). https://www.w3.org/TR/owl-primer/

Neo4j, Inc. (s. f.). Cypher manual. https://neo4j.com/docs/cypher-manual/current/introduction/

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The definitive guide to Scrum. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

Anexo A. Guía rápida de ejecución y evidencias

A.1 Protégé

1. Instalar o abrir Protégé Desktop compatible con OWL 2.
2. Usar File > Open y seleccionar ontologia/BrujulaAcademica.owl.
3. Revisar la jerarquía de clases y las pestañas Object properties / Data properties; comprobar dominios y rangos.
4. Configurar y ejecutar un razonador compatible con OWL 2 (p. ej. HermiT), revisar consistencia y tipos inferidos de resourcePythonBasics y resourceSQLLab.
5. Tomar capturas de: clases y propiedades, restricciones, individuos, estado del razonador y clases inferidas. La captura debe ser producida por el equipo en la aplicación.

A.2 Neo4j

1. Crear una base de prueba en Neo4j Desktop o Neo4j Aura y abrir Neo4j Browser.
2. Copiar y ejecutar primero el bloque 0 (restricciones) y bloque 1-2 (nodos y relaciones) de neo4j/brujula_academica.cypher.
3. Ejecutar cada consulta separadamente. Proporcionar los parámetros studentId = STU-001 y advisorId = ADV-001 cuando aplique.
4. Capturar el grafo, conteo de nodos/relaciones y resultados de las consultas 1, 2 y 3. Registrar versión de Neo4j y ajustes hechos.
5. No adjuntar capturas simuladas: registrar salidas reales de ejecución.